DBMS MINI PROJECT REPORT

Insurance Claim Document Verification & Settlement Tracking System

Database Management Systems – Mini Project

Submitted by

Student Name	SRN
Pavan E Naik	PES2UG24CS343
Pavankumar H	PES2UG24CS345

1. Title

Insurance Claim Document Verification & Settlement Tracking System

2. Students – SRN and Name

Sl. No.	Student Name	SRN
1	Pavan E Naik	PES2UG24CS343
2	Pavankumar H	PES2UG24CS345

3. Abstract

This project is a web-based system for managing insurance claims from submission to settlement. Customers can log in, submit claims against their policies and upload supporting documents. Claim officers can approve or reject claims and verify uploaded documents with remarks. Authorized staff can update settlement payment status, while customers can view only their own claims, documents and settlement details. MySQL stores structured project data, and MongoDB stores document-verification event history.

4. Tech Stack

Technology	Purpose
Next.js, React, TypeScript	Frontend and API routes
Tailwind CSS	Interface styling
MySQL + mysql2/promise	Relational database and SQL access
MongoDB	Document verification event history
jose (JWT), bcryptjs	Authentication and password hashing
VS Code, MySQL Workbench, MongoDB Compass	Development and database tools
Git and GitHub	Version control

5. SQL – Database Commands

Database: insurance_claim_db. MySQL inspection showed eight base tables and one view.

Object	Type
users, customers, insurance_policies, claims	Base tables
claim_documents, claim_reviews, claim_status_history, settlements	Base tables
claim_details_view	View

Representative SQL commands

```
USE insurance_claim_db;
SHOW FULL TABLES;

SELECT claim_status, COUNT(*) AS total_claims
FROM claims GROUP BY claim_status;

SELECT claim_status, COUNT(*) AS total_claims
FROM claims GROUP BY claim_status HAVING COUNT(*) > 1;

SELECT claim_id, claim_number, claim_amount
FROM claims
WHERE claim_amount > (SELECT AVG(claim_amount) FROM claims);
```

```
SELECT c.claim_id, c.claim_number, ip.policy_id, cu.customer_id
FROM claims c
JOIN insurance_policies ip ON c.policy_id = ip.policy_id
JOIN customers cu ON ip.customer_id = cu.customer_id;

SELECT COUNT(*) AS total_claims, SUM(claim_amount) AS total_amount,
       AVG(claim_amount) AS average_amount, MAX(claim_amount) AS highest_amount
FROM claims;
```

Other SQL concepts examined include views, indexes, foreign keys, constraints, stored procedures, triggers and transactions. The application uses a transaction when recording a claim review and updating claim status. Add exact screenshots/results for any procedure, trigger or index you intend to claim as implemented.

6. NoSQL / Vector Database Commands

The project uses MongoDB (not a vector database). Database: insurance_claim_nosql; collection: document_verification. It stores upload and verification events with document_id, claim_id, status changes, remarks and timestamps.

```
db.document_verification.aggregate([
  { $group: { _id: "$new_status", total: { $sum: 1 } } }
]);
```

Observed aggregation result: REJECTED = 1, VERIFIED = 3, PENDING = 2 (6 records total).

7. Next.js – Project Tree Structure

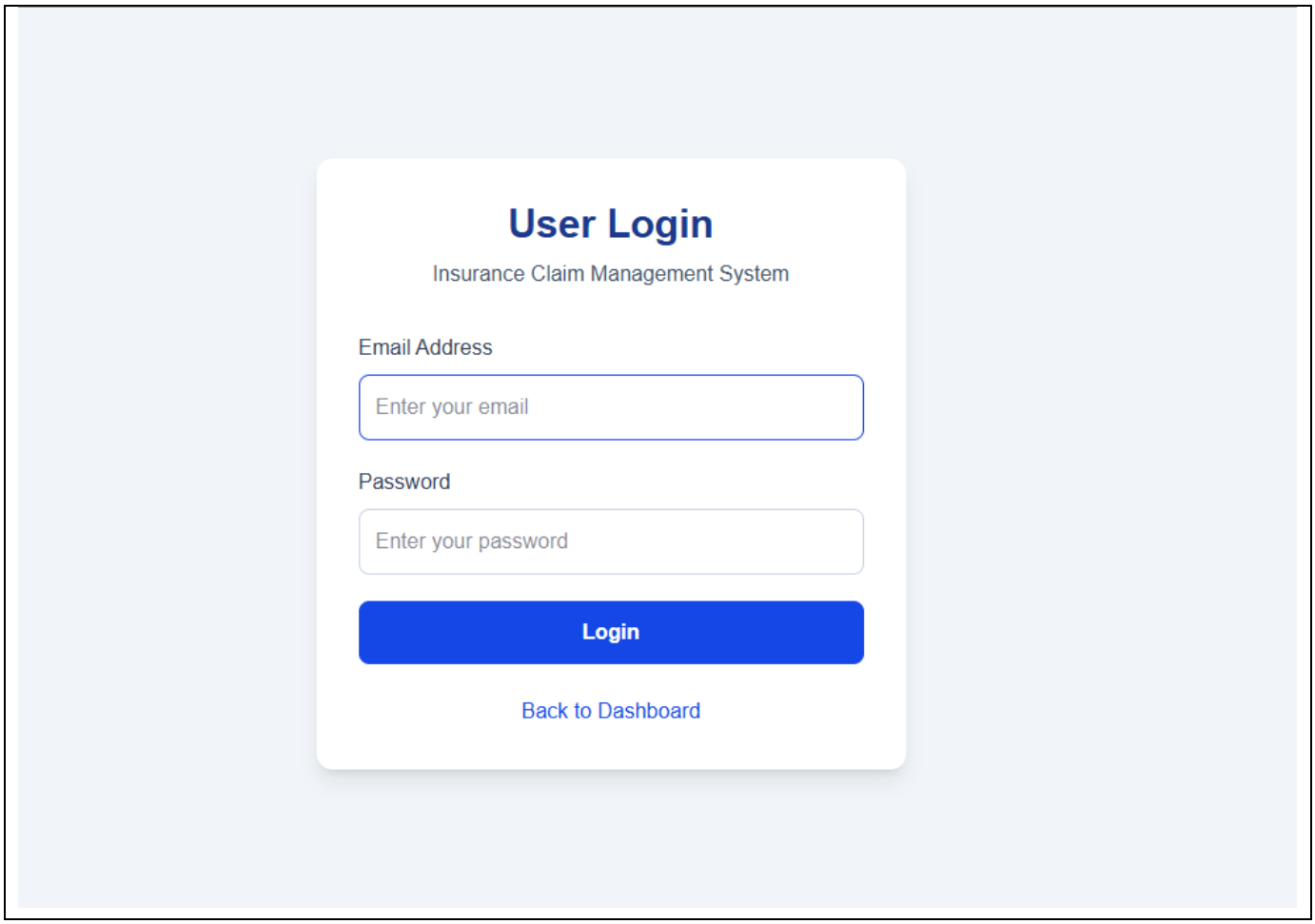
```
insurance-claim-system/
├── app/
│   ├── api/
│   │   ├── auth/login/route.ts
│   │   ├── claims/route.ts
│   │   ├── documents/route.ts
│   │   ├── documents/[documentId]/route.ts
│   │   └── settlements/route.ts
│   ├── claims/new/page.tsx
│   ├── claims/page.tsx
│   ├── documents/page.tsx
│   └── settlements/page.tsx
├── lib/
│   ├── db.ts
│   └── auth.ts
├── .gitignore
├── package.json
└── check-db.js
```

Private uploads and .env.local are kept out of GitHub using .gitignore.

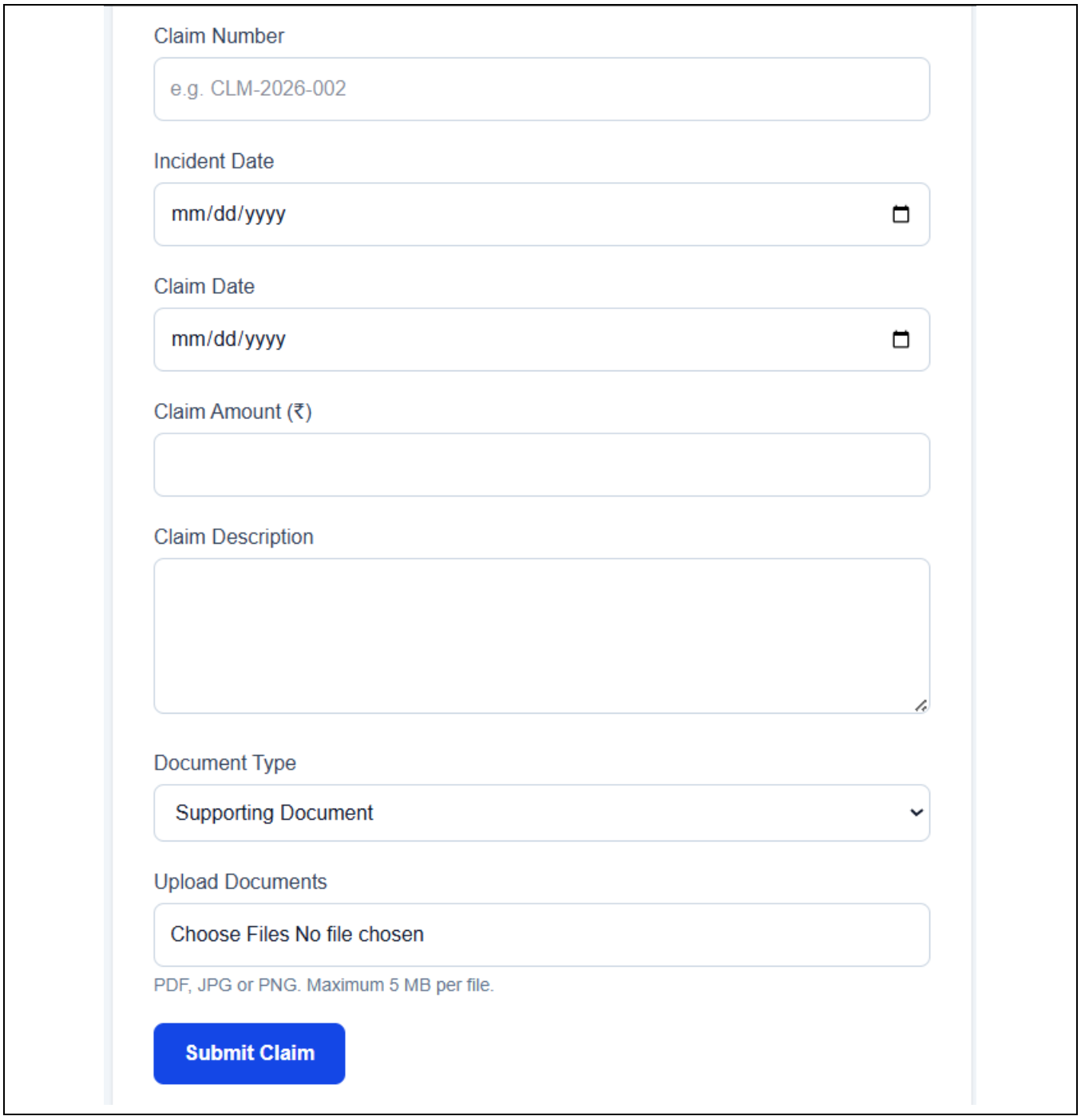
8. Sample Screenshots – Code and Demo

Insert a few clear screenshots below. Hide passwords, tokens and private customer information.

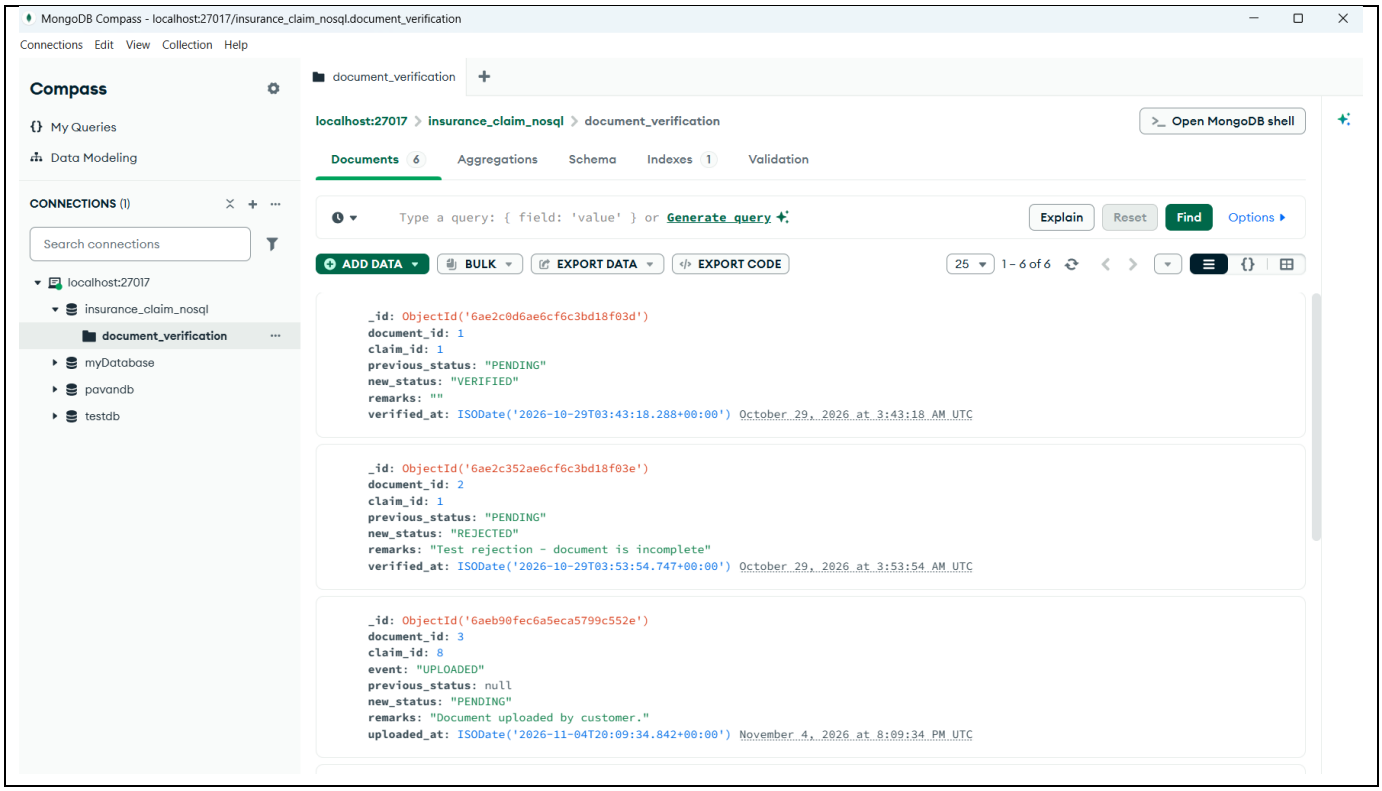
Customer login and claims page



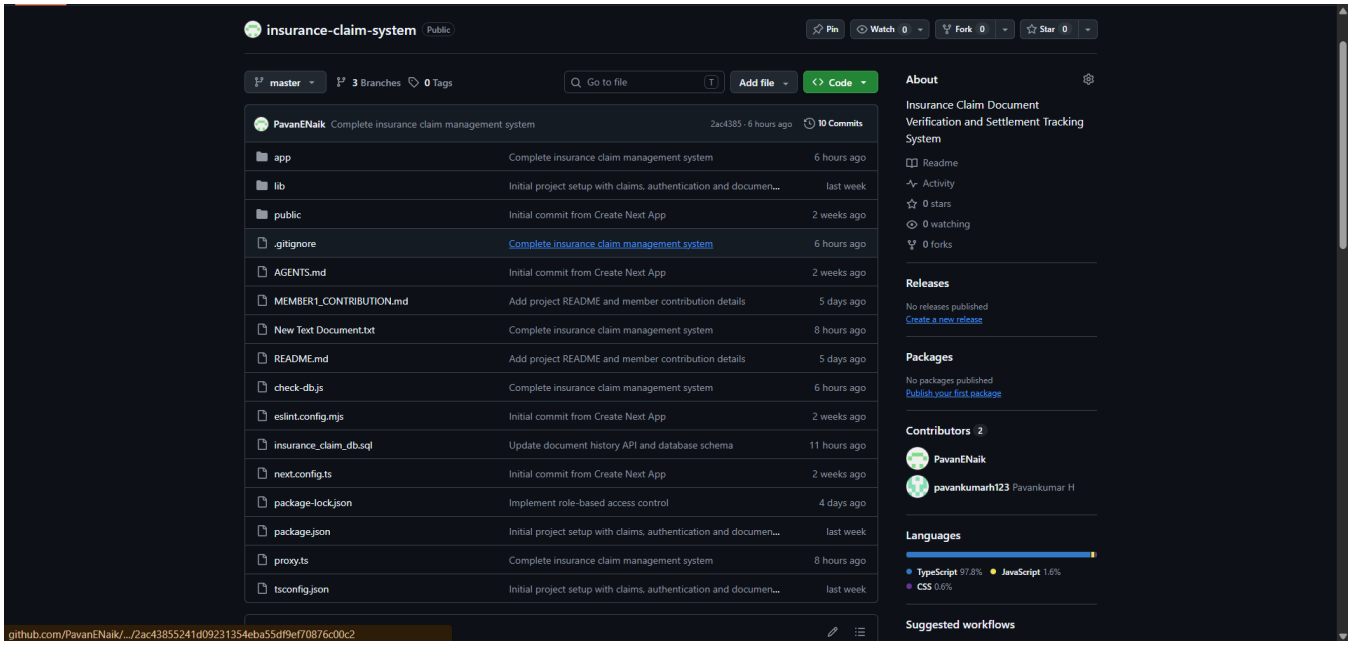
Claim submission and document upload



MongoDB aggregation



Github repository



9. Evaluation Rubrics – Completion Status

Area	Status
Next.js frontend and API routes	Completed
MySQL connection and relational tables	Working; 8 tables and 1 view observed

Customer login and role-based access	Implemented and tested
Claim submission and staff review	Working in application tests
Document upload and verification	Working in application tests
Settlement tracking and status updates	Working in application tests
MongoDB event history and aggregation	Working; aggregation verified
SQL querying (JOIN, GROUP BY, HAVING, subquery, aggregates)	Checked
GitHub commit and push	Completed
Report screenshots and student details	To be filled before submission

Conclusion

The system combines a Next.js application with MySQL for relational claim data and MongoDB for document-verification history. It supports customer claim submission, staff review, document verification and settlement tracking with role-based access. The application was tested and its latest changes were pushed to GitHub.

Github link : https://github.com/PavanENaik/insurance-claim-system.git